

© The Author(s), under exclusive license to APress Media, LLC, part of Springer Nature 2023
M. Frisbie, *Building Browser Extensions*
https://doi-org.ezproxy.sfpl.org/10.1007/978-1-4842-8725-5_13

13. Networking

Matt Frisbie¹
(1) California, CA, USA

As you wade deeper into browser extension development, you will quickly realize that there are some interesting idiosyncrasies when it comes to sending network requests. Although browser extensions and web pages use the same APIs and network layers to send requests, there are fundamental differences that you will need to surmount when building a well-formed browser extension. This chapter will cover some core areas of extension networking: dispatching network requests, authentication, and browser extension networking APIs.

Comparing Websites and Extensions

To begin this chapter, let's set the stage by comparing a traditional website and a browser extension.

Category	Websites	Browser Extensions (MV3)
Origin	A website has a predictable origin and a user-provided domain.	Browser extensions are automatically assigned an origin via the extension ID.
APIs	Websites can use XMLHttpRequest or fetch() anywhere.	Background scripts can only use fetch().
Remote assets	Remote assets can be served from the same origin without configuring a cross-origin policy.	Remote assets must be loaded with a cross-origin policy correctly configured. Scripts cannot be executed from remote origins.
Page types	All web pages on a website can send requests and authenticate in consistent fashion.	Popup/options pages, content scripts, and background scripts all have different restrictions for sending requests, for example, content scripts are subject to the host

Category	Websites	Browser Extensions (MV3)
		page’s cross-origin restrictions, but popups are not.
Server requests	The website can send same-origin requests to the backend.	If a browser extension uses a backend server, the requests will <i>always</i> be cross-origin.
Authentication	No restrictions: the website can use cookie auth, JWT auth, or OAuth anywhere.	Not all forms of authentication work everywhere, for example, service workers can’t use cookie authentication.
Long-running requests	Any long-running requests will stay alive while a tab is open.	Long-running requests in background service workers, popups, and content scripts may all be unexpectedly terminated, for example, a popup is closed or a service worker is terminated.
Cross-browser	The website origin remains consistent across browsers.	The browser extension origin is different across browsers: <code>chrome-extension://extensionID</code> vs. <code>moz-extension://uuid</code> .

Networking Architecture

When it comes to sending network requests, different components of browser extensions are better than others for certain tasks. How you dispatch your network requests will depend on the nature of your browser extension. This section will cover some things to consider when designing your extension.

Options Pages

Options pages are nearly identical to a traditional website, and therefore they are a natural choice for many developers when building a browser extension. If your entire user interface is built inside options pages, you will not have any problem using any form of authentication, be it cookie auth, JSON Web Token (JWT) auth, or OAuth. Unless the options page is rendered as a modal (`open_in_tab=false`), it will enjoy the same lifecycle as a browser tab, so long-running requests can be safely dispatched from an options page without risk of early termination.

The options page is a good option as a supplement for browser extensions that wish to rely heavily on a background script, but also wish to execute

long-running requests. The background script can kick open an options tab as a vehicle for the long-running requests.

Popup and Devtools Pages

Popup and devtools pages are nearly identical in nature to options pages, but with one primary difference: they are expected to be closed early and often. Thus, they are not a good vehicle for long-running network requests. Otherwise, they are good choices for authentication and general-purpose network request.

Content Scripts

Content scripts are a particularly interesting tool for sending network requests. Because they are running on the host page, they are subject to the same cross origin restrictions. This means talking to your own server will likely require delegating that request to the background script. However, because content script requests are treated as host page requests, they can use the host page's cookies – meaning you can send requests as the authenticated user.

Authenticating inside the content script is tricky, as the host page can see anything placed into the DOM or in a shared API. Furthermore, long-running network requests are obviously dependent on the host page remaining open; if the tab closes or the user navigates away, the request or web-socket will be abruptly terminated.

Tip The authentication spoofing ability is incredibly powerful! It usually requires some reverse engineering of the host page, but it allows you to automate authenticated actions that the host page is allowed to perform. For example, suppose some web page shows an authenticated user's shopping cart with 100 items in it. Each item has a "remove item" button that triggers a network request to `/cart/remove` with a JSON payload like `{"item_id": 123}`, and you wish to empty the cart. Instead of automating 100 button clicks, you can instead extract the required `item_id` values and the content script can send those 100 authenticated requests directly.

Background Scripts

The transition from manifest v2 to manifest v3 was especially challenging for background scripts, as the network request paradigm changed in substantial and breaking ways. Previously, the background script existed as a persistent headless web page, meaning that cookie auth, long-running requests, and auth dialog windows were allowed. Now, all these things have materially changed:

- Cookie authentication is still technically possible, but the cookies need to be passed in from a rendered HTML page and manually provided, which is very annoying and impractical – or impossible, in the case of HttpOnly cookies.
- The browser does not consider inflight requests when determining if a service worker is idle, so long-running requests are subject to early termination.

- Auth dialog windows cannot make use of `window.open()`, they must use the identity API.

Background scripts are still an excellent and very capable tool, but with manifest v3 they are certainly more limited. Extensions that use JWT or OAuth authentication and do not rely on long-running requests will be able to seamlessly utilize background service worker networking without issue.

Pinning an Extension ID

In the context of extension networking, it is useful to be able to predict and control the origin of a browser extension. When developing locally, the browser will assign your extension an ID automatically; when uploading to the Chrome web store, it will assign your extension an entirely different ID. For developers that wish to have a consistent ID, this is problematic.

Fortunately, it is possible to pin your extension’s ID ahead of time to make it consistent and predictable. Doing so involves the following steps:

- 1.**Upload the initial version of your extension to the Chrome Web Store.** The store will generate a public key for your extension, which in turn is used to generate the extension ID. Your local browser will generate the same extension ID if it is provided that public key (Figure 13-1).
- 2.**Retrieve the public key from the web store (Figure 13-2).**
- 3.**Add the public key to your manifest under the “key” property.** Strip out the BEGIN/END lines and all the whitespace and pass it in as one big string (Figure 13-3).
- 4.**Load the extension locally.** Your local browser should generate an identical extension ID to the Chrome Web Store (Figure 13-4).

CautionKeep in mind that the production extension and the local extension having the same ID means the browser will consider them to be the same. To prevent collision conflicts, ensure you only have the production or local extension installed at any given time.

This process is shown below:

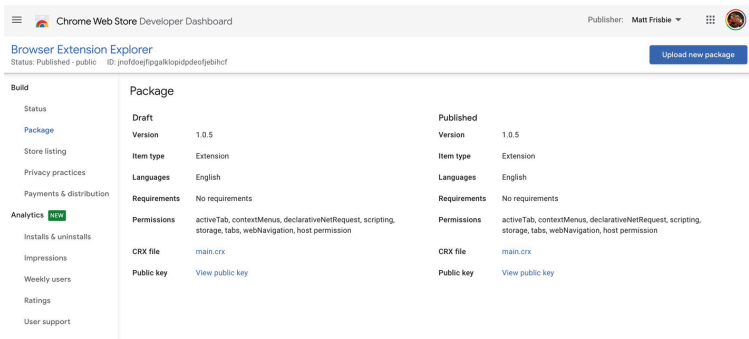


Figure 13-1 The Chrome Web Store package page including the public key link. Note the ID beginning with “jnof”

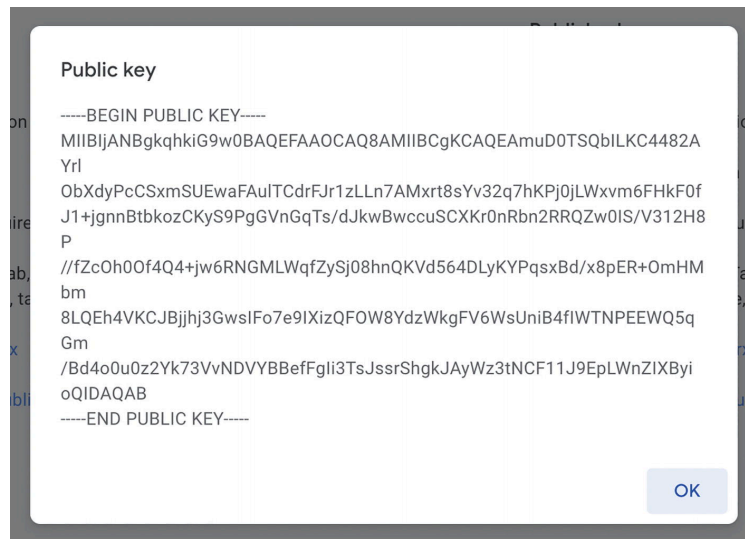


Figure 13-2 The revealed public key



Figure 13-3 The flattened public key passed to the manifest

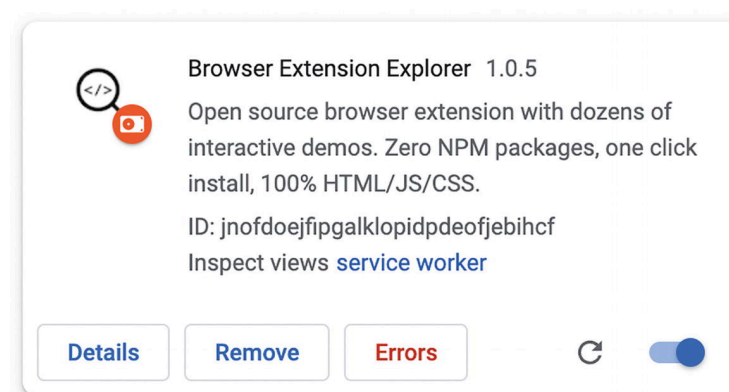


Figure 13-4 The extension loaded locally. Note the ID beginning with “jnof”

Authentication Styles

The web wouldn't have gotten far without the ability to authenticate. Browser extensions can use the same foundational technologies as websites to authenticate. However, due to the innate architecture of extensions, the way you go about authenticating must be carefully considered. This section discusses different modalities of authentication.

No Authentication

Browser extensions don't necessarily need to authenticate. Often, a clever application of a small portion of the WebExtensions API is all that is needed to make an extension useful. What's more, when a content script interacts with the raw DOM, the browser extension's ability to interact with authenticated sessions is totally disjoint from any concept of a logged in state. In other words, the host website can do the heavy lifting

of logging in the user, the browser extension can wait for a logged in state, and then the content script can read from and write to the DOM.

Some APIs allow you to take advantage of native browser identity separation without managing authentication. For example, the storage API allows you to use `chrome.storage.sync`, which is attached to the user's browser account and will be synced to their account, or

`chrome.storage.local`, which is localized to the current browser instance.

Content Script Spoofing

As mentioned earlier in the chapter, network requests from a content script are treated the same as network requests from a host page script. Therefore, if a website uses HttpOnly cookie authentication, a content script will be able to piggyback on that authenticated state and send requests that will automatically include those cookies.

Of course, a well-built website will also include a CSRF token, but since the CSRF token is usually provided in a consistent location such as inside a `<form>`, a content script can inspect the DOM, extract the CSRF token, and pass that along with the outgoing request.

Cookie Authentication

Cookie authentication is technically possible for browser extension user interfaces, but I don't recommend it. This style of authentication works best when the web page is rendered directly by a server, and for browser extensions this is never the case. Furthermore, a cookie-based authenticated state can't be directly accessed by the background service worker. Overall, because it involves so much unnecessary overhead to implement, I recommend you avoid cookie authentication unless you have a particularly compelling reason to do so.

Json Web Token Authentication

Token-style authentication works well for app-based authentication where cookies aren't practical. For browser extensions, token authentication schemes like JWT (JSON Web Token) are ideal. Popups, options pages, devtools pages, and background scripts can all directly authenticate with a server and share the authentication token between extension components without issue. Content scripts are able to use JWTs, but the host page's cross-origin policy will likely block requests to the server, so sending requests indirectly via the background service worker is the preferred workaround.

OAuth and OpenID

With the `chrome.identity` API, browser extensions have first-class support for OAuth and OpenID. This API is not supported inside content scripts, but it can be effectively deployed anywhere else in a browser extension. For developers that do not wish to implement a backend server,

authorization and authentication via OAuth or OpenID is extremely practical.

The `chrome.identity` API is spearheaded by Google Chrome, but it enjoys at least partial support in all chromium browsers and Firefox. In Google Chrome, browser extensions enjoy additional identity API methods including a streamlined OAuth authorization flow (more on this later in the chapter).

NoteSafari currently has no support for the identity API.

OAuth, OpenID, and the Identity API

NoteThis section assumes you have basic knowledge of how the OAuth2 protocol works. For an introduction to OAuth2, Auth0 has an excellent writeup on it: <https://auth0.com/intro-to-iam/what-is-oauth-2/>.

The ability to delegate authentication and authorization to a third-party platform is incredibly useful, but implementing this in browser extensions is tricky. Consider some of the challenges involved:

- Delegating a login to a third-party platform requires the ability to open a trusted interface to collect the credentials. Background service workers cannot utilize `window.open()`.
- Configuring a platform to support OAuth requires knowledge of the extension ID ahead of time.
- OAuth and OpenID utilize the same underlying protocol, which includes a redirect URL. Browser extensions can only render inside private URLs with an extension protocol.

Fortunately, the identity API solves all these problems by providing a flexible set of tools for implementing delegated authentication and authorization.

OAuth API Methods

There are two ways of using OAuth in an extension:

- `chrome.identity.getAuthToken()` allows you to natively authenticate. This method allows you to skip providing a redirect URL and making the authorization token request. Instead, you simply provide the required values inside the `oauth2` manifest property and call this method. The browser will kick open the OAuth dialog and the method's callback will be passed the OAuth token. This is *by far* the simplest way of implementing OAuth in an extension, but it is only available inside Google Chrome. **This requires a “Chrome app” client ID.**
- `chrome.identity.launchWebAuthFlow()` is the more generalized method for using OAuth2. It is cross-browser (works on Firefox, Edge, etc.) and cross-platform (supports OAuth2 with Facebook, Github, etc.). It is much more labor intensive, as it requires you to manually implement each of the OAuth2 steps.

OAuth Redirect URLs

To solve the OAuth redirect URL problem, the browser supports a special URL that will direct the auth flow back to the extension. To access this URL, you can use the `chrome.identity.getRedirectURL()` method. The Chrome documentation describes its behavior:

This method enables auth flows with non-Google identity providers by launching a web view and navigating it to the first URL in the provider's auth flow. When the provider redirects to a URL matching the pattern `https://<app-id>.chromiumapp.org/*`, the window will close, and the final redirect URL will be passed to the callback function.

In other words, the browser gives this URL special treatment in order to support authentication in the context of browser extensions. All chromium browsers and Firefox support this method.

Tip You can see this special URL handling logic in the chromium source code:

<https://chromium.googlesource.com/chromium/chromium/+master/chrome/browser/extensions/api/identity>

Configuring the Authorization Platform

The first step in setting up OAuth for a chrome extension is to configure the authorization platform to provide access to the extension. This involves configuring the OAuth consent screen and generating a client id (Figures 13-5, 13-6, 13-7, 13-8, and 13-9). The consent screen is what will be shown in the popup when the user begins the OAuth authentication flow; the client ID is the string that uniquely identifies the extension and allows it to engage with the authorization platform (Figure 13-10).

Note This process will be slightly different for various authorization platforms and OAuth flows, but it will always provide you with a client ID.

The following screenshots demonstrate some of these steps when configuring Google OAuth.

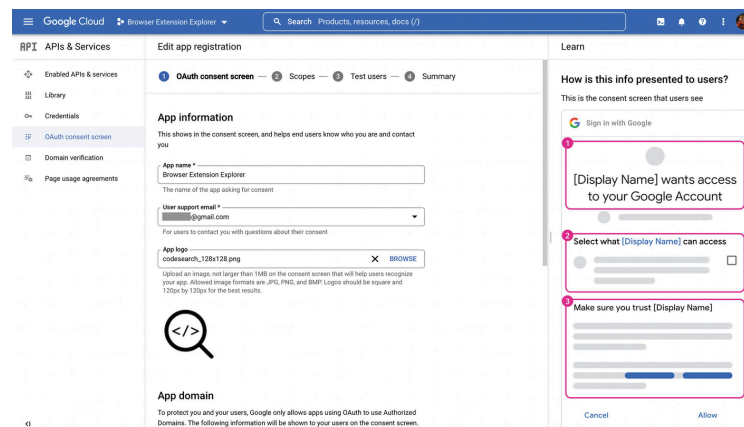


Figure 13-5 Configuring the OAuth consent screen in the Google Cloud dashboard

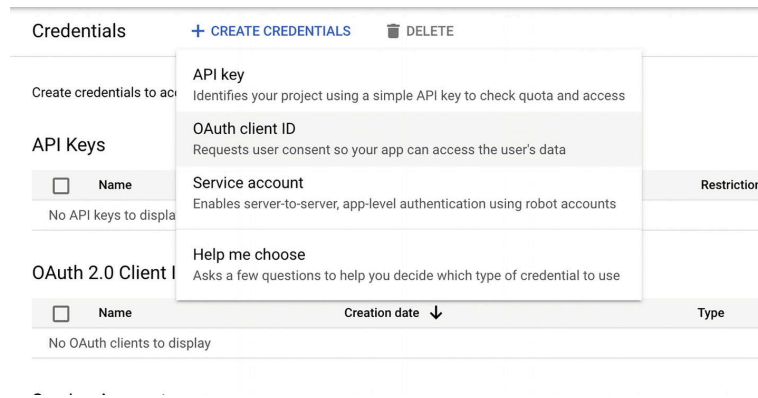


Figure 13-6 Creating OAuth client ID credentials for a browser extension in the Google Cloud dashboard

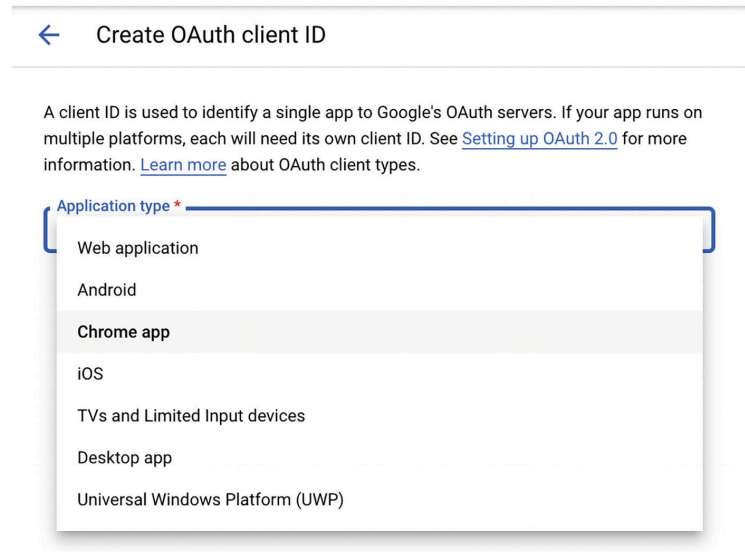


Figure 13-7 Google's options for an OAuth client ID

Not all platforms will support this distinction. In the case of Google OAuth, both “Chrome app” and “Web application” can be used for browser extensions.

- **Chrome app** means you are using the manifest `oauth2` manifest field and the `getAuthToken()` method. This will also authenticate the user in the active Google Chrome browser profile, which may not be desirable.
- **Web application** means you are using the `launchWebAuthFlow()` method. This allows you to authenticate separately from the Google Chrome browser profile.

Browser Extension Explorer

Search Products, resources, docs (/)

←

Create OAuth client ID

A client ID is used to identify a single app to Google's OAuth servers. If your app runs on multiple platforms, each will need its own client ID. See [Setting up OAuth 2.0](#) for more information. [Learn more](#) about OAuth client types.

Application type *

Chrome app

Name *

Browser Extension Explorer

The name of your OAuth 2.0 client. This name is only used to identify the client in the console and will not be shown to end users.

Application ID *

jnofdoejfpgalklopidpdeoefjebihcf

Last part of your app's Chrome Web Store URL.

Note: It may take 5 minutes to a few hours for settings to take effect

CREATE

CANCEL

Figure 13-8 Configuring a Chrome app OAuth client ID

APIs & Services

← Client ID for Web application

DOWNLOAD JSON

RESET SECRET

Enabled APIs & services

Library

Credentials

OAuth consent screen

Domain verification

Page usage agreements

Name *

Browser Extension Explorer (Web)

The name of your OAuth 2.0 client. This name is only used to identify the client in the console and will not be shown to end users.

The domains of the URIs you add below will be automatically added to your [OAuth consent screen](#) as [authorized domains](#).

Authorized JavaScript origins

For use with requests from a browser

+ ADD URI

Authorized redirect URIs

For use with requests from a web server

URIs 1 *

https://jnofdoejfpgalklopidpdeoefjebihcf.chromiumapp.org/

+ ADD URI

Note: It may take 5 minutes to a few hours for settings to take effect

SAVE

CANCEL

Figure 13-9 Configuring a Web application client ID

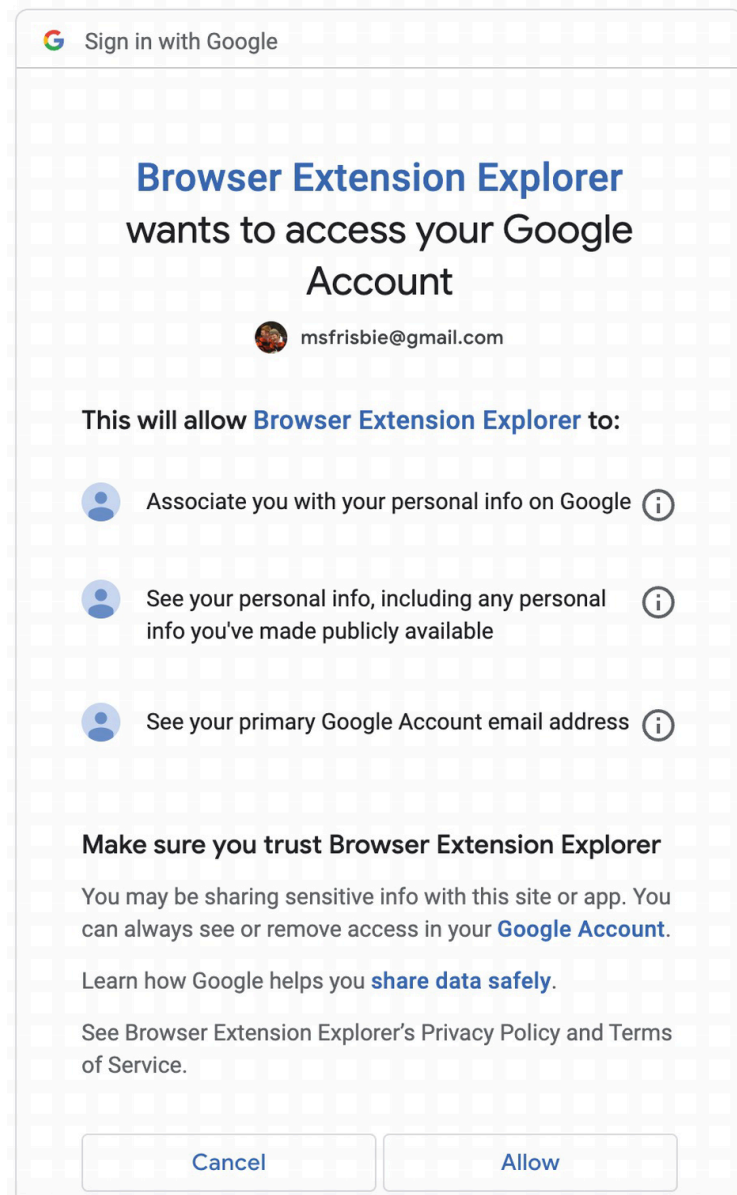


Figure 13-10 The working OAuth consent screen

Google will eventually provide you with your client ID in a downloadable JSON. A dummy example is shown here:

client_secret.json

```
{
  "installed": {
    "client_id": "<some_id>.apps.googleusercontent.com",
    "project_id": "browser-extension-explorer",
    "auth_uri": "https://accounts.google.com/o/oauth2/auth",
    "token_uri": "https://oauth2.googleapis.com/token",
    "auth_provider_x509_cert_url": "https://www.googleapis.com/oauth2/v1/certs"
  }
}
```

Other platforms will provide you the client ID in a different fashion and format, but ultimately this is what must make it into your browser extension to enable OAuth authentication.

Additional Help

Setting up OAuth is extremely confusing. I've hand-selected some links that walk you through setting it up in various ways:

- https://developer.chrome.com/docs/extensions/mv3/tut_oauth/
- <https://blog.plasmo.com/p/firebase-chrome-extension>

OAuth and OpenID Examples

When it comes to OAuth for extensions, there really isn't a good substitute for getting your hands dirty with actual working code. The following examples demonstrate slightly different ways of configuring and triggering an OAuth flow. **These examples are provided in source code, but they will not work without some legwork on your end. You will need to configure your own OAuth provider information such as client IDs to make them work correctly.**

Tip The book's companion extension, *Browser Extension Explorer*, includes three working OAuth examples. Visit buildingbrowserextensions.com to install it.

Google OAuth with `getAuthToken()`

This example uses Google Chrome's native OAuth faculties to authenticate the user. Clicking the toolbar icon triggers the OAuth flow, and a successful authentication will log some basic fields. Some things to note:

- The permissions requested include `identity`, which gives access to the identity API, and `identity.email`, which allows the extension to easily access the email and Gaia ID of the user account signed into the current profile via `chrome.identity.getProfileUserInfo()`.
- `getAuthToken()` is idempotent. After the user authenticates, it will cache the OAuth token. If called a when the user is authenticated, no OAuth dialog will appear: the method will retrieve the token from the cache and pass it to the callback. The `interactive` option controls whether it will attempt to launch the OAuth dialog if the user is *not* authenticated. If `false`, it will exit and throw an error if not authenticated. Otherwise, the OAuth dialog will open.
- Note that nowhere in this flow are any URLs other than what is provided in the manifest. The redirect URL and OAuth endpoints are all automatically handled by the browser!

```
{
  "name": "MVX",
  "version": "0.0.1",
  "manifest_version": 3,
  "action": {},
  "background": {
    "service_worker": "background.js",
    "type": "module"
  },
  "permissions": ["identity", "identity.email"],
  "oauth2": {
    "client_id": "YOUR_CLIENT_ID",
```

```

"scopes": [
  "https://www.googleapis.com/auth/userinfo.email",
  "https://www.googleapis.com/auth/userinfo.profile"
]
}
}

```

Example 13-1a manifest.json

```

chrome.action.onClicked.addListener(() => {
  chrome.identity.getAuthToken(
    {
      interactive: true,
    },
    (token) => {
      if (token) {
        chrome.identity.getProfileUserInfo(
          { accountStatus: "ANY" },
          (info) => console.log(info)
        );
      }
    }
  );
});
// Console output:
// { "email": <google_email>, "id": <google_gaia_id> }

```

Example 13-1b background.js

Google OpenID with launchWebAuthFlow()

Note This section assumes you have basic knowledge of how the OpenID Connect protocol works. For an introduction to OpenID, Auth0 has an excellent writeup on it: <https://auth0.com/intro-to-iam/what-is-openid-connect-oidc/>.

This example implements a Google OpenID Connect flow to authenticate the user using `launchWebAuthFlow()`. Clicking the toolbar icon triggers the OpenID flow, and a successful authentication will log the OpenID payload. Some things to note

- Compared to the previous example, we no longer have a need for the `identity.email` permission since we are not using the `getProfileUserInfo()` method.
- Note that the client ID is being passed inside JavaScript rather than provided in the manifest.
- With `launchWebAuthFlow()`, you must now manually provide the redirect URL that the browser will use to redirect back into your extension's callback.
- `launchWebAuthFlow()` has an `interactive` control which behaves identically to that of `getAuthToken()`.
- OpenID requires specific query string parameters in the initial URL and returns a JSON web token (JWT). This example unpacks the JWT so you can see the data packaged inside.

```

{
  "name": "MVX",
  "version": "0.0.1",

```

```

"manifest_version": 3,
"action": {},
"background": {
  "service_worker": "background.js",
  "type": "module"
},
"permissions": ["identity"]
}

```

Example 13-2a manifest.json

```

chrome.action.onClicked.addListener(() => {
  const clientId = "YOUR_CLIENT_ID";
  const extensionRedirectUri = chrome.identity.getRedirectURL();
  const nonce = Math.random().toString(36).substring(2, 15);
  const authUrl = new URL("https://accounts.google.com/o/oauth2/v2/auth");
  // Define fields for OpenID
  authUrl.searchParams.set("client_id", clientId);
  authUrl.searchParams.set("response_type", "id_token");
  authUrl.searchParams.set("redirect_uri", extensionRedirectUri);
  authUrl.searchParams.set("scope", "openid profile email");
  authUrl.searchParams.set("nonce", nonce);
  authUrl.searchParams.set("prompt", "consent");
  chrome.identity.launchWebAuthFlow(
    {
      url: authUrl.href,
      interactive: true,
    },
    (redirectUrl) => {
      if (redirectUrl) {
        // The ID token is in the URL hash
        const urlHash = redirectUrl.split("#")[1];
        const params = new URLSearchParams(urlHash);
        const jwt = params.get("id_token");
        // Parse the JSON Web Token
        const base64Url = jwt.split(".")[1];
        const base64 = base64Url.replace("-", "+").replace("_", "/");
        const token = JSON.parse(atob(base64));
        console.log(token);
      }
    }
  );
});
// Console output:
// {
//   "iss": "https://accounts.google.com",
//   "azp": "...",
//   "aud": "...",
//   "sub": "...",
//   "email": "XXXXX@gmail.com",
//   "email_verified": true,
//   "nonce": "...",
//   "name": "Matt Frisbie",
//   "picture": "...",
//   "given_name": "Matt",
//   "family_name": "Frisbie",
//   "locale": "en",
//   "iat": ...,
//   "exp": ...,
//   "jti": "..."
// }

```

Example 13-2b background.js

Manual Github OAuth with launchWebAuthFlow()

This example implements a Github OAuth2 flow to authenticate the user using `launchWebAuthFlow()`. Clicking the toolbar icon triggers the Github OAuth2 flow, and a successful authentication will log the Github profile payload. Some things to note

- This example does all the legwork of a full OAuth flow: launching the authentication dialog, collecting the authorization code, using that to fetch the access token, and finally performing an OAuth-authenticated request.
- The Github OAuth dialog passes the initial authorization code by appending it to the redirect URL, which here is plucked out inside the callback.
- Github's OAuth provides you with a client secret that must be included with the request to fetch the access token.

```
{
  "name": "MVX",
  "version": "0.0.1",
  "manifest_version": 3,
  "action": {},
  "background": {
    "service_worker": "background.js",
    "type": "module"
  },
  "permissions": ["identity"]
}
```

Example 13-3a manifest.json

```
chrome.action.onClicked.addListener(() => {
  const clientId = "YOUR_CLIENT_ID";
  const extensionRedirectUri = chrome.identity.getRedirectURL();
  const authUrl = new URL("https://github.com/login/oauth/authorize");
  authUrl.searchParams.set("client_id", clientId);
  authUrl.searchParams.set("redirect_uri", extensionRedirectUri);
  chrome.identity.launchWebAuthFlow(
    {
      url: authUrl.href,
      interactive: true,
    },
    async (redirectUrl) => {
      if (redirectUrl) {
        const queryString = new URL(redirectUrl).search;
        const params = new URLSearchParams(queryString);
        const code = params.get("code");
        const authUrl = new URL("https://github.com/login/oauth/access_token");
        authUrl.searchParams.append("client_id", clientId);
        authUrl.searchParams.append("redirect_uri", extensionRedirectUri);
        authUrl.searchParams.append(
          "client_secret",
          "YOUR_CLIENT_SECRET"
        );
        authUrl.searchParams.append("code", code);
        const response = await fetch(authUrl, {
```

```

        method: "POST",
        headers: {
            Accept: "application/json",
        },
    });
    const accessTokenData = await response.json();
    const r = await fetch("https://api.github.com/user", {
        headers: {
            Authorization: "Bearer " + accessTokenData.access_token,
        },
    });
    console.log(await r.json());
    }
}
);
});
// Console output:
// {
//   "login": "msfrisbie",
//   "id": ...,
//   "node_id": "...",
//   "avatar_url": "...",
//   "name": "Matt Frisbie",
//   "blog": "https://www.mattfriz.com",
//   "location": "Chicago, IL",
//   "bio": "Software engineer, bestselling author",
//   "twitter_username": "mattfriz",
//   "url": "https://api.github.com/users/msfrisbie",
//   ...
// }

```

Example 13-3b background.js

Networking APIs

Browser extensions are granted access to some powerful APIs that can be used to inspect and modify traffic flow inside the browser. These methods are the reason ad blocking extensions are so effective. This section will examine three APIs:

- The `webNavigation` API allows you to observe top-level browser navigation events in very granular detail.
- The `webRequest` API allows you to intercept and modify traffic in a blocking fashion, meaning that for any network request, the extension can inject a JavaScript function that may or may not modify or cancel the request entirely. The “blocking” designator indicates that the network request will wait for the JavaScript function to return before continuing.
- The `declarativeNetRequest` API allows you to build rulesets that instruct the browser how it should natively handle network requests. These rules may tell the browser to modify or block a request. This API is the successor to the `webRequest` API and is considerably less powerful in several ways. This API is only available in manifest v3.

NoteIn manifest v3, support for `webRequest` is fragmented. For example, Firefox will continue to support this API, whereas Chromium browsers are removing support.

The webNavigation API

The `webNavigation` API allows an extension to see navigation events from the browser. The API allows an extension to add event handler for the following navigation lifecycle events:

- `onBeforeNavigate`
- `onCommitted`
- `onCompleted`
- `onCreatedNavigationTarget`
- `onDOMContentLoaded`
- `onErrorOccurred`
- `onHistoryStateUpdated`
- `onReferenceFragmentUpdated`
- `onTabReplaced`

Figure 13-11 shows the flow of events.

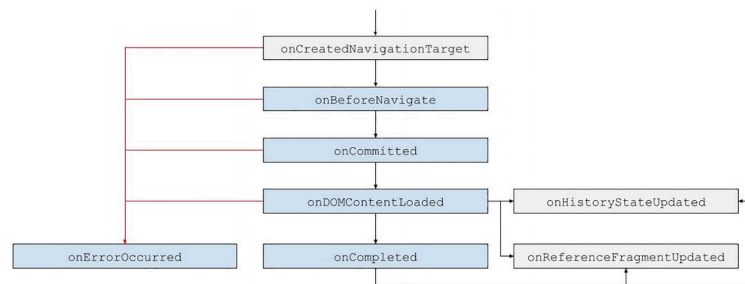


Figure 13-11 The `webNavigation` event flow

Note Details of each of these events can be found in the MDN documentation: <https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/API/webNavigation>.

Most of the time, extensions will only care to set handlers for the primary lifecycle events fired by normal browser navigation:

1. `onBeforeNavigate`
2. `onCommitted`
3. `onDOMContentLoaded`
4. `onCompleted`

The following simple extension will log to the extension console any time a tab in the browser visits a new URL. Load the extension, open a new tab, and navigate to any website to see navigation events in real time.

```

{
  "name": "MVX",
  "version": "0.0.1",
  "manifest_version": 3,
  "background": {
    "service_worker": "background.js",
    "type": "module"
  },
  "permissions": ["webNavigation"]
}
  
```

Example 13-4a manifest.json

```
chrome.webNavigation.onCompleted.addListener((details) => {
  console.log(details);
});
// Example console log:
// {
//   "documentId": "7D0F7EBBCB0020C74DA89F2D36C461B4",
//   "documentLifecycle": "active",
//   "frameId": 0,
//   "frameType": "outermost_frame",
//   "parentFrameId": -1,
//   "processId": 1040,
//   "tabId": 173953269,
//   "timeStamp": 1661549345594.7412,
//   "url": "https://en.wikipedia.org/wiki/Main_Page"
// }
```

Example 13-4b background.js

The webRequest API

The `webRequest` API allows browser extensions to inspect and modify network requests events from a web page. The API allows an extension to add event handler for the following request lifecycle events:

- `onActionIgnored`
- `onAuthRequired`
- `onBeforeRedirect`
- `onBeforeRequest`
- `onBeforeSendHeaders`
- `onCompleted`
- `onErrorOccurred`
- `onHeadersReceived`
- `onResponseStarted`
- `onSendHeaders`

Figure 13-12 shows the flow of events.

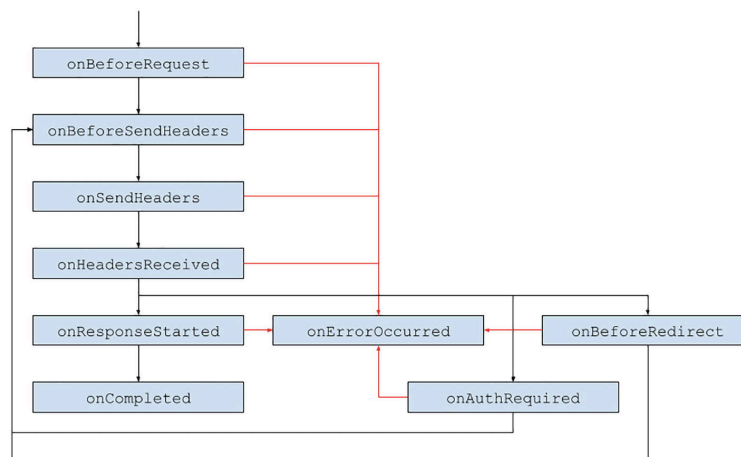


Figure 13-12 The `webRequest` event flow

Note Details of each of these events can be found in the MDN documentation: <https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/API/webRequest>.

Modifying Requests

If you merely wish to inspect the request lifecycle event data, only the `webRequest` permission is needed. If, however, you wish to potentially modify these requests, you will need an additional `webRequestBlocking` permission. This grants you the ability to perform the following tasks:

- The extension can **cancel a request** inside the `onBeforeRequest`, `onBeforeSendHeaders`, or `onAuthRequired` blocking handler.
- The extension can **redirect the request** inside the `onBeforeRequest` or `onHeadersReceived` blocking handler.
- The extension can **modify the request headers** inside the `onBeforeSendHeaders` blocking handler.
- The extension can **modify the response headers** in the `onHeadersReceived` blocking handler.
- The extension can **modify the request's authentication credentials** in the `onAuthRequired` blocking handler.

The following simple example will block all image requests on Wikipedia. Keep in mind that you must also request URL permissions if you want to inspect or modify requests on that origin.

Note The `webRequest` API and manifest v3 lack support in Google Chrome, so this example is written in manifest v2. I recommend testing it out in Firefox.

```
{
  "name": "MVX",
  "version": "0.0.1",
  "manifest_version": 2,
  "background": {
    "scripts": ["background.js"]
  },
  "permissions": [
    "webRequest",
    "webRequestBlocking",
    "<all_urls>"
  ]
}
```

Example 13-5a manifest.json

```
chrome.webRequest.onBeforeRequest.addListener(
  () => {
    return {
      cancel: true,
    };
  },
  {
    urls: [
      "*/www.wikipedia.org/portal/wikipedia.org/assets/img/*"
    ],
  },
  ["blocking"]
);
```

Example 13-5b background.js

Fragmentation

The browser extension space is headed in a bizarre direction. At the time this book was written, it appears that Chromium browsers have decided to end support for `webRequest` in manifest v3, but Firefox may continue to support the API in manifest v3. It is unclear where the space will end up, but at the moment I would not recommend building an extension that relies directly upon this API.

The declarativeNetRequest API

The `declarativeNetRequest` API (DNR) is billed as the successor to the `webRequest` API. The motivation for this shift was to eliminate the blocking JavaScript handlers in favor of a more efficient declarative model that is implemented by the browser. Now, instead of performing modifications inside a JavaScript function, you will supply the “instructions” for each matching request as a JSON config object.

For most extensions, DNR is an effective replacement for `webRequest`. However, ad blocker quality will suffer for two primary reasons:

- Browsers enforce a global cap on the number of rules that extensions can define for `declarativeNetRequest`. Modern ad blockers define on the order of 100,000+ rules, and ad blocker extensions are undoubtedly going to run into this cap limit quickly.
- Modern ad blockers rely on sophisticated logic inside the blocking `webRequest` handlers. Not all of this logic can be translated into the comparatively simplistic `declarativeNetRequest` rules, so ad blocker extensions will not be as effective at fighting ads.

Permissions

The `declarativeNetRequest` API can be used with three different permissions:

- The `declarativeNetRequest` permission allows your extension to define DNR rules, but you will still need to request host permissions to apply rules on specific origins.
- The `declarativeNetRequestWithHostAccess` permission can be used instead of the `declarativeNetRequest` permission. It allows you to block or upgrade requests on any origin without requesting host permissions. Redirects or header modification will still require specific host permissions.
- The `declarativeNetRequestFeedback` permission can be added as a supplement to the previous two. With it, your extension can access functions and events which return information on rules matched.

Rule and Ruleset Structure

All DNR rules take the form of a JSON object. Only one rule will ever be applied per request, per extension. Each rule object is structured as follows:

- The `id` must be a unique positive integer.

- The `priority` is an optional integer that can be used to control which rule is applied in scenarios where multiple rules match a request.
- The `condition` is an object that describes when this rule should be applied. With it, you can include or exclude domains, domain types, tab IDs, and resource types.
- The `action` is an object that describes what this rule does. It includes a type, which can be `block`, `redirect`, `allow`, `upgradeScheme`, `modifyHeaders`, or `allowAllRequests`, and various properties which configure how the rule should behave.

DNR rules are defined in one of two ways:

- **Static rulesets** are arrays of rule objects in JSON files. Ruleset files can be provided in the manifest or added programmatically. They can also be enabled or disabled on demand. Static rulesets can only ever exist as immutable files packaged with the extension. The total number of static rules is limited by the browser.
- **Dynamic rules** are rule objects that are exclusively programmatically created. They can be added, removed, modified, and enabled or disabled. A subset of dynamic rule are **session rules**, which are dynamic rules that are not persisted across browsers. The total number of dynamic rules is limited by the browser.

Whether static or dynamic, a DNR rule is treated the same by the browser.

Note The `declarativeNetRequest` API has quite a bit of depth to it. Refer to the Google Chrome documentation for details on all its features:

<https://developer.chrome.com/docs/extensions/reference/declarativeNetRequest>.

Static Rulesets

Let's examine static rulesets by examining the following simple extension that can block images on Wikipedia:

```
{
  "name": "MVX",
  "version": "0.0.1",
  "manifest_version": 3,
  "background": {
    "service_worker": "background.js",
    "type": "module"
  },
  "declarative_net_request": {
    "rule_resources": [
      {
        "id": "ruleset_1",
        "enabled": true,
        "path": "rules_1.json"
      }
    ]
  },
  "action": {},
  "permissions": ["declarativeNetRequest"],
  "host_permissions": [
    "*//*.wikipedia.org/*",
  ]
}
```

```

    "**://*.wikipedia.org/"
  ]
}

```

Example 13-6a manifest.json

```

const RULESET_ID = "ruleset_1";
chrome.action.onClicked.addListener(async () => {
  const enabled_rulesets =
    await chrome.declarativeNetRequest.getEnabledRulesets();
  if (enabled_rulesets.includes(RULESET_ID)) {
    chrome.declarativeNetRequest.updateEnabledRulesets({
      disableRulesetIds: [RULESET_ID],
    });
  } else {
    chrome.declarativeNetRequest.updateEnabledRulesets({
      enableRulesetIds: [RULESET_ID],
    });
  }
  console.log("Toggled ruleset");
});

```

Example 13-6b background.js

```

[
  {
    "id": 1,
    "priority": 1,
    "action": {
      "type": "block"
    },
    "condition": {
      "domains": ["wikipedia.org", "wikimedia.org"],
      "resourceTypes": ["image"]
    }
  }
]

```

Example 13-6c rules_1.json

Load this extension and navigate to `wikipedia.org`. The initial extension state will enable the ruleset. You will notice that images do not load. Click the extension's toolbar button to toggle the ruleset. When you reload Wikipedia again, images should load normally.

You may notice that a new `_metadata` directory has appeared inside your local development folder (Figure 13-13). As the name suggests, the binary file inside the directory is how the browser keeps track of active rulesets. Feel free to ignore it but ensure you do not commit it to version control or add it to a production build.

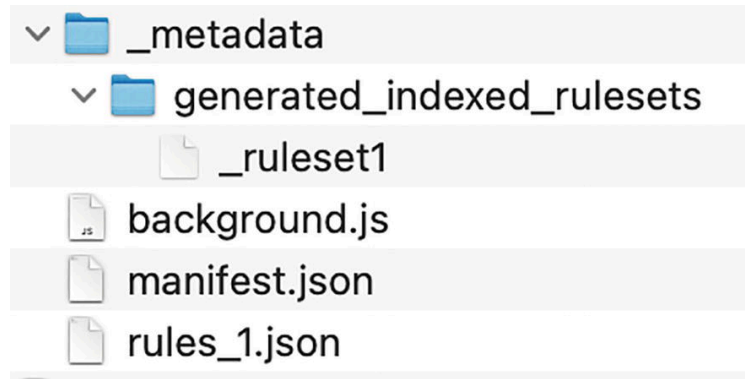


Figure 13-13 Automatically generated indexed ruleset binary

Dynamic Rules

Dynamic rules are very similar to static rulesets. In similar fashion, they can be queried, added, and removed on demand. In this example, let's instead write a dynamic redirect rule:

```
{
  "name": "MVX",
  "version": "0.0.1",
  "manifest_version": 3,
  "background": {
    "service_worker": "background.js",
    "type": "module"
  },
  "action": {},
  "permissions": ["declarativeNetRequest"],
  "host_permissions": ["*://*.wikipedia.org/*", "*://*.wikimedia.org/*"]
}
```

Example 13-7a manifest.json

```
const RULE_ID = 1;
const RULE_1 = {
  id: RULE_ID,
  priority: 1,
  action: {
    type: "redirect",
    redirect: {
      url: "https://upload.wikimedia.org/wikipedia/commons/thumb/b/b1/Hot_dog_with_mustard.png/1920px-H",
    },
  },
  condition: {
    domains: ["wikipedia.org", "wikimedia.org"],
    resourceTypes: ["image"],
  },
};

chrome.action.onClicked.addListener(async () => {
  const dynamic_rules = await
    chrome.declarativeNetRequest.getDynamicRules();
  if (dynamic_rules.find((rule) => rule.id === 1)) {
    chrome.declarativeNetRequest.updateDynamicRules({
      removeRuleIds: RULE_ID,
    });
  } else {
    chrome.declarativeNetRequest.updateDynamicRules({
      addRules: [RULE_1],
    });
  }
});
```

```
});  
}  
console.log("Toggled rule");  
});
```

Example 13-7b background.js

Load this extension and navigate to `wikipedia.org`. You will notice that images load normally. Click the extension's toolbar button and reload Wikipedia again. All images on Wikipedia should now be a hot dog.

Summary

In this chapter, we explored a variety of modalities in which browser extensions can send network requests. We reviewed all the different components and how they can and should send network requests. Next, we discussed authentication strategies, including in-depth coverage on how to implement OAuth in browser extensions. Finally, we discussed how to use the major WebExtensions APIs that can inspect and manipulate traffic in the browser.

In the next chapter, we will cover how to develop an extension locally, publish it on extension marketplaces, and deploy updates.